

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192372087
Name	Atmakuri Sai Mrudula

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I **Atmakuri Sai Mrudula (192372087)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **A. Sai Mrudula**

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

A media platform needs an HDFS-based storage solution for storing large amounts of videos, images, audio files, and metadata. The system must maintain data availability above 99.9%, the replication factor must not exceed 3, and storage growth must remain cost efficient.

Proposed Design

The proposed solution uses HDFS with a replication factor of 3, rack-aware replica placement, NameNode High Availability, and suitable storage policies.

Main Components

- **HDFS NameNode:** Maintains metadata and manages file access.
- **Standby NameNode:** Provides high availability if the Active NameNode fails.
- **DataNodes:** Store media files as HDFS blocks.
- **Replication Factor 3:** Maintains three copies of important data blocks.
- **Rack-Aware Placement:** Places replicas across different racks to protect against rack failure.
- **Compression:** Reduces storage requirements for suitable media metadata and supporting files.
- **Storage Policies:** Frequently accessed data can use faster storage, while older media can be moved to lower-cost storage.
- **Monitoring:** Monitors DataNode health and under-replicated blocks.

Constraint Justification

99.9% Availability

NameNode High Availability prevents the NameNode from becoming a single point of failure. Replication across DataNodes ensures that data remains accessible when an individual node fails.

Replication Factor ≤ 3

A replication factor of **3** provides good fault tolerance while satisfying the constraint that replication cannot exceed 3.

Cost Efficiency

Storage costs can be controlled by:

- Using replication factor 3 only where required.
- Compressing suitable data.
- Moving older or less frequently accessed data to lower-cost storage.
- Removing unnecessary duplicate files.
- Monitoring storage utilization.



Conclusion

An HDFS cluster with replication factor 3, rack-aware placement, NameNode High Availability, and cost-aware storage management can provide high availability while controlling storage growth.

2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

A MapReduce workflow must process large amounts of log data within a 20-minute batch window while operating with a limited number of cluster nodes.

Proposed Design

The system uses parallel MapReduce processing with properly sized input splits, multiple mapper tasks, efficient combiners, and an appropriate number of reducers.

Processing Stages

1. Store log files in HDFS.
2. Divide logs into balanced input splits.
3. Assign mapper tasks to available nodes.
4. Parse and filter log records.
5. Use a Combiner to reduce intermediate data.
6. Shuffle and partition the mapper output.
7. Assign data to reducers.
8. Generate the final log analytics output.

Task Distribution

Since the number of nodes is limited, the system should avoid creating excessive tasks.

- Use balanced input splits.
- Run multiple mapper tasks across available nodes.
- Use data locality whenever possible.
- Use a Combiner to reduce network traffic.
- Choose the reducer count according to the dataset size and available resources.
- Avoid unnecessary processing and disk I/O.

Example

For a log dataset divided into six balanced splits:

Node 1 → Mapper 1 + Mapper 2

Node 2 → Mapper 3 + Mapper 4

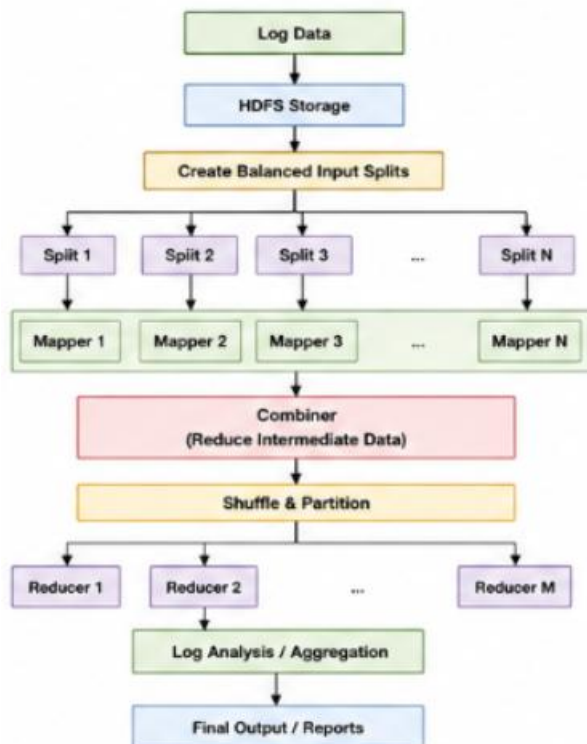
Node 3 → Mapper 5 + Mapper 6

The mapper results are then shuffled and distributed among reducers.

20-Minute Constraint

To meet the 20-minute batch window:

- Balance input splits.
- Maximize parallel task execution.
- Use data locality.
- Reduce shuffle data using Combiners.
- Avoid reducer data skew.
- Monitor task execution time.
- Allocate available cluster resources efficiently.



Conclusion

A balanced MapReduce design with parallel mappers, combiners, controlled reducers, and data locality can process log data efficiently within the 20-minute batch constraint even with limited cluster nodes.

3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

A YARN cluster supports ETL, reporting, and machine learning jobs. The scheduler must provide fairness, give priority to ETL jobs, and recover from node failures.

Proposed Scheduler Policy

The recommended approach is a **queue-based Capacity Scheduler** with priorities and resource limits.

ETL Queue

ETL receives higher priority because it prepares data required by reporting and machine learning workloads.

Machine Learning Queue

Machine learning jobs receive sufficient resources but should not consume the entire cluster.

Reporting Queue

Reporting workloads can use available resources without blocking high-priority ETL jobs.

Fairness

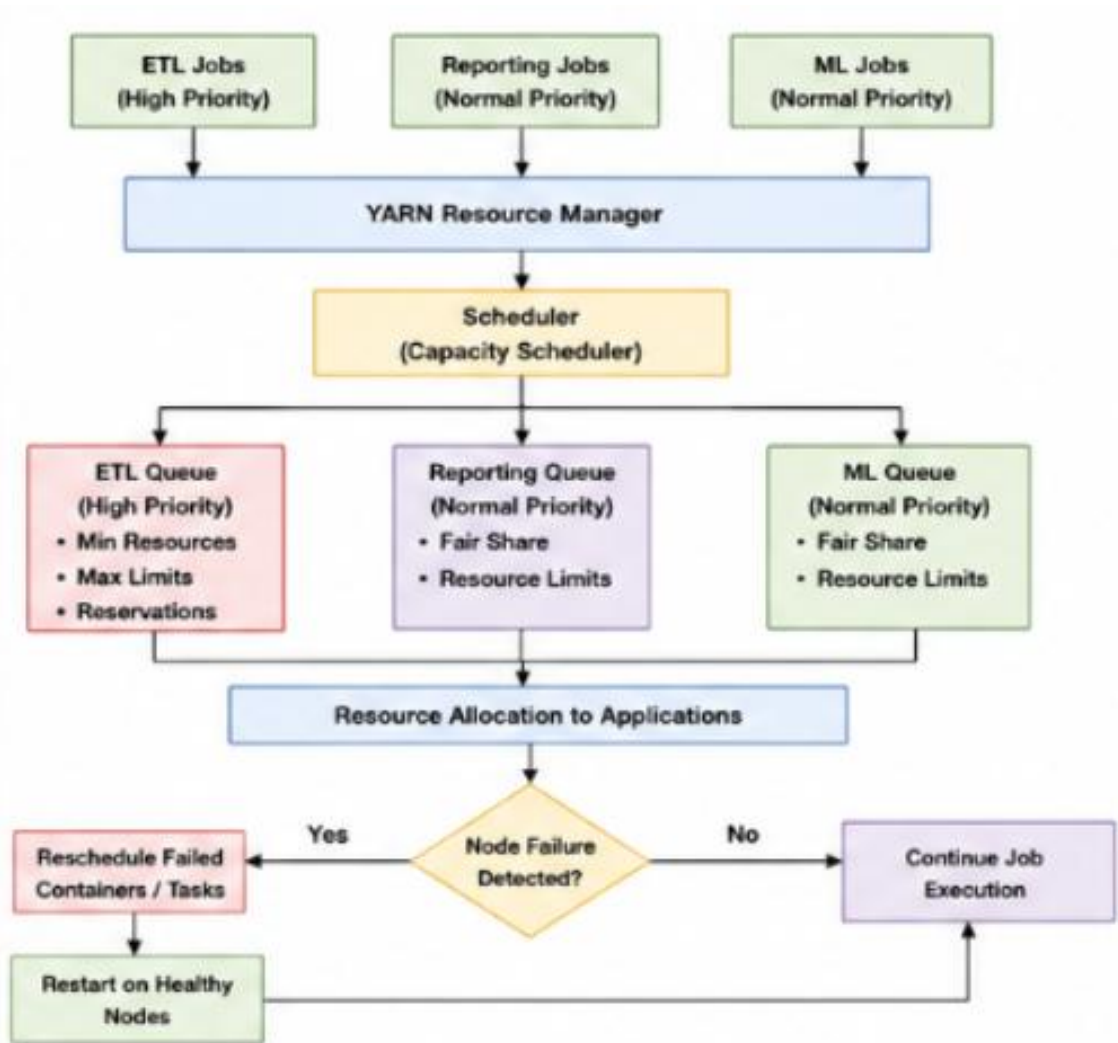
Fairness can be maintained using:

- Queue capacity limits.
- User quotas.
- CPU and memory limits.
- Application priorities.
- Maximum resource allocation per queue.

Node Failure Recovery

If a NodeManager fails:

1. ResourceManager detects the failure.
2. Failed containers are identified.
3. Tasks are rescheduled on healthy nodes.
4. Lost tasks restart.
5. Queue resources are recalculated.



Conclusion

A queue-based scheduler with high ETL priority, fair resource allocation, quotas, and automatic task recovery satisfies all three constraints while preventing one workload from monopolizing the cluster.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication.

Proposed Architecture

The proposed solution uses Apache NiFi/ETL for ingestion, HDFS for distributed storage, a metadata/catalog layer, and deduplication mechanisms.

Main Components

SAN

Stores enterprise files and application data.

NAS

Provides shared file-based storage.

Operational Databases

Contain structured business transactions and application data.

Apache NiFi / ETL

Collects data from multiple sources, transforms it, validates it, and transfers it to the big data platform.

HDFS

Provides scalable distributed storage.

Deduplication Layer

Uses unique record IDs, checksums, timestamps, and database constraints to prevent duplicate records.

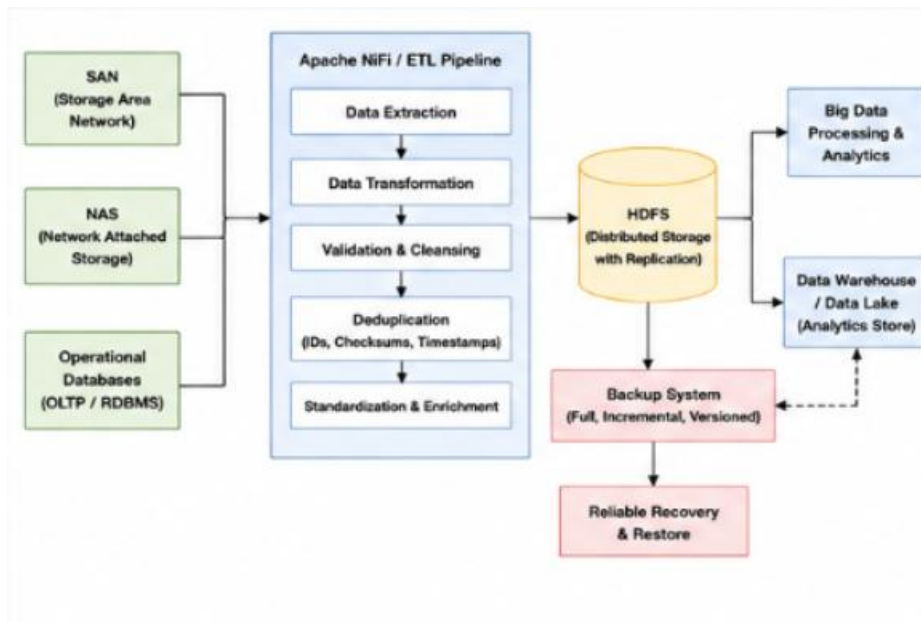
Backup System

Maintains independent and versioned backups for recovery.

Backup Reliability

Backup reliability can be improved by:

- Maintaining multiple backup versions.
- Using incremental and full backups.



- Monitoring backup completion.
- Storing backups separately from production data.
- Regularly testing restoration.

Minimal Duplication

Duplicate data can be controlled using:

- Unique record IDs.
- Duplicate detection.
- Database UNIQUE constraints.
- Idempotent ingestion.
- Upsert operations.

Constraint Justification

The architecture supports multiple data sources while NiFi/ETL provides centralized ingestion and transformation. HDFS provides scalable storage, while deduplication and independent backups improve data quality and reliability.

Conclusion

Using NiFi/ETL + HDFS + deduplication + versioned backup provides a reliable and scalable enterprise data ingestion architecture.

5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

An organization needs an end-to-end Hadoop ecosystem solution under the constraints of:

- Secure data movement
- High availability
- Integration with Apache NiFi or ETL pipelines

Proposed Architecture

The proposed system integrates Apache NiFi, HDFS, YARN, MapReduce, Hive, and backup mechanisms.

Main Components

1. Apache NiFi

Collects data from databases, files, applications, and external sources.

2. Secure Data Transfer

Data movement should use secure communication such as encryption and authenticated connections.

3. HDFS

Stores large datasets using distributed blocks and replication.

4. NameNode High Availability

Uses Active and Standby NameNodes to avoid a single point of failure.

5. YARN

Manages CPU and memory resources across the cluster.

6. MapReduce / Processing Engine

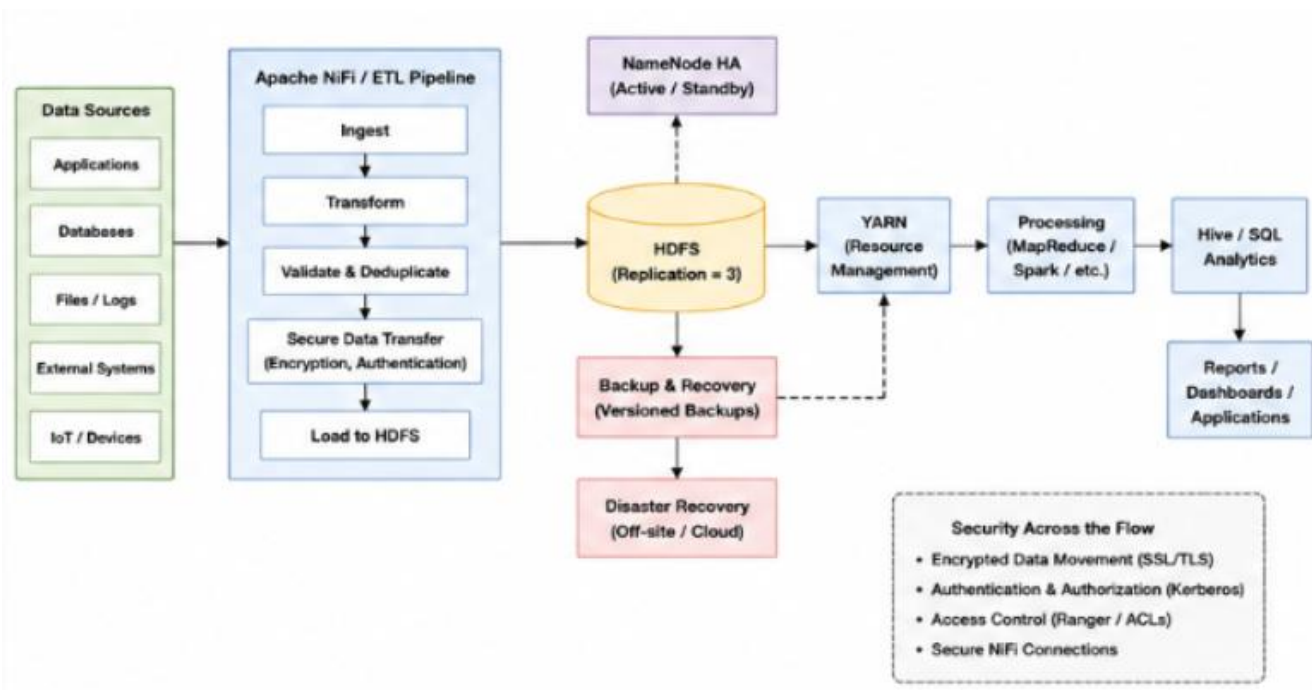
Processes large datasets in parallel.

7. Hive / Analytics

Provides SQL-based analysis over stored data.

8. Backup and Recovery

Maintains versioned backups and recovery points.



High Availability

High availability is achieved using:

- Active and Standby NameNodes.
- HDFS data replication.
- Multiple DataNodes.
- Automatic task recovery.
- Monitoring and failure detection.
- Reliable backup and recovery.

Secure Data Movement

Security can be maintained through:

- Encrypted data transmission.
- Authentication.
- Authorization.
- Secure database connections.
- Controlled access to HDFS.
- Secure NiFi data flows.

Constraint Justification

Constraint	Design Solution
Secure data movement	Encryption, authentication and controlled access
High availability	HDFS replication + NameNode HA
Data ingestion	Apache NiFi / ETL
Large-scale storage	HDFS
Resource management	YARN
Parallel processing	MapReduce
Data analysis	Hive / analytics layer
Data recovery	Versioned backups

The proposed Hadoop ecosystem integrates Apache NiFi/ETL, secure data transfer, HDFS, YARN, MapReduce, Hive, and backup systems. HDFS replication and NameNode High Availability provide fault tolerance, while secure ingestion protects data during movement. NiFi provides flexible integration with enterprise data sources, making the architecture secure, scalable, highly available, and suitable for large-scale cloud and big-data applications.